```
    public ConfigureCurrencyOptions(ICurrencyProvider currencyProvider)
    {
        _currencyProvider = currencyProvider;                              ❶
    }

    public void Configure(CurrencyOptions options)                         ❷
    {
        options.Currencies = _currencyProvider.GetCurrencies();            ❸
    }
}
```

❶ You can inject services that are available only after the DI is completely configured.

❷ Configure is called when an instance of IOptions<CurrencyOptions> is required.

❸ Uses the injected service to load the values

All that remains is to register the implementation in the DI container. As always, order is important, so if you want `ConfigureCurrencyOptions` to run after binding to configuration, you must add it after configuring your `OptionsBuilder<T>`:

```
builder.Services.AddOptions<CurrencyOptions>()
    .BindConfiguration("Currencies");
builder.AddSingleton
    <IConfigureOptions<CurrencyOptions>, ConfigureCurrencyOptions>();
```

TIP The order in which you configure your options matters. If you want to always run your configuration last, after all other configuration methods, you can use the `PostConfigure()` method on `OptionsBuilder`, or the `IPostConfigureOptions` interface. You can read more about this approach on my blog at http://mng.bz/mVj4.

With this configuration, when `IOptions<CurrencyOptions>` is injected into an endpoint or service, the `CurrencyOptions` object is first bound to the `"Currencies"` section of your